

B Academy
RO

Manual

Flexbox



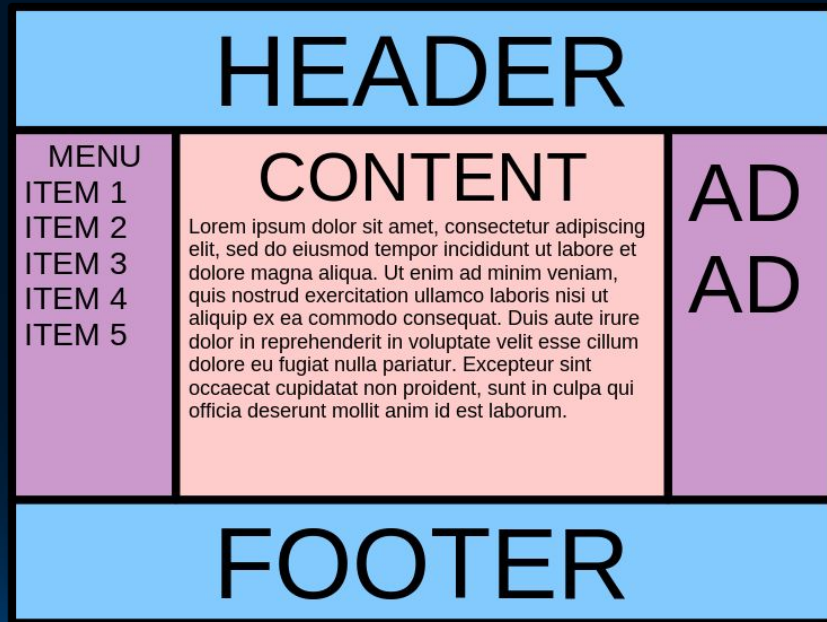


flexbox

Flexbox allows control over:

- Size
- Order
- Alignment of elements across multiple axes
- Distribution of free space between elements
- and more

- ★ Ease in creating complex layouts
- ★ Automatic alignment
- ★ Responsiveness



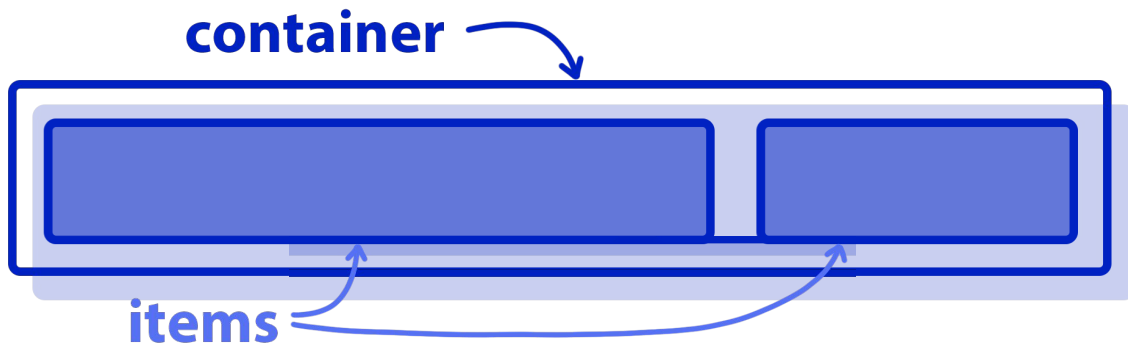
flex container and items

The container is given the property:

```
display: flex;
display: inline-flex;
```

Style properties are applied to the **elements** inside the container.

The elements adjust to the width of their content and are arranged in a row by default.



[Demo](#)

flex-direction

This establishes the main-axis, thus defining the direction flex items are placed in the flex container. Flexbox is (aside from optional wrapping) a single-direction layout concept. Think of flex items as primarily laying out either in horizontal rows or vertical columns.

main axis ~~horizontal~~

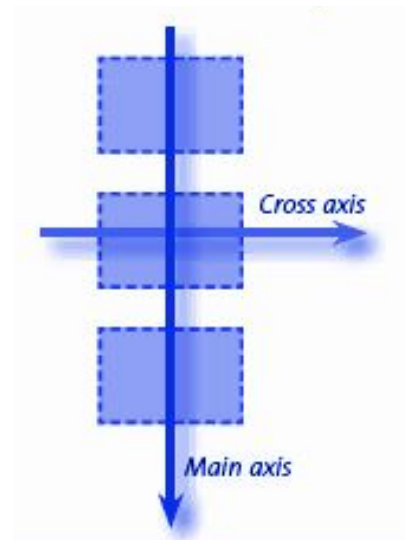
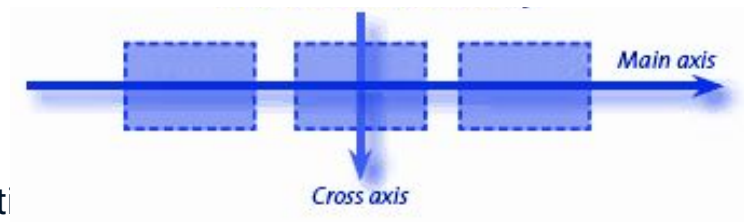
cross axis ~~vertical~~

Default:

`flex-direction: row;`

To arrange elements in a column:

`flex-direction: column;`



Demo

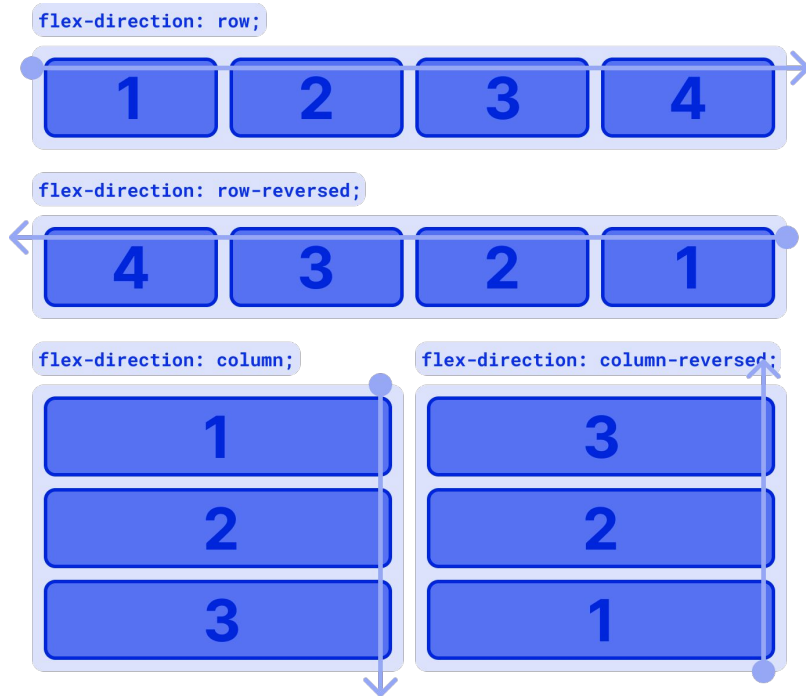
flex-direction: reversed

Demo

Using Reverse Direction:

```
flex-direction: row-reversed;
```

```
flex-direction: column-reversed;
```



flex order

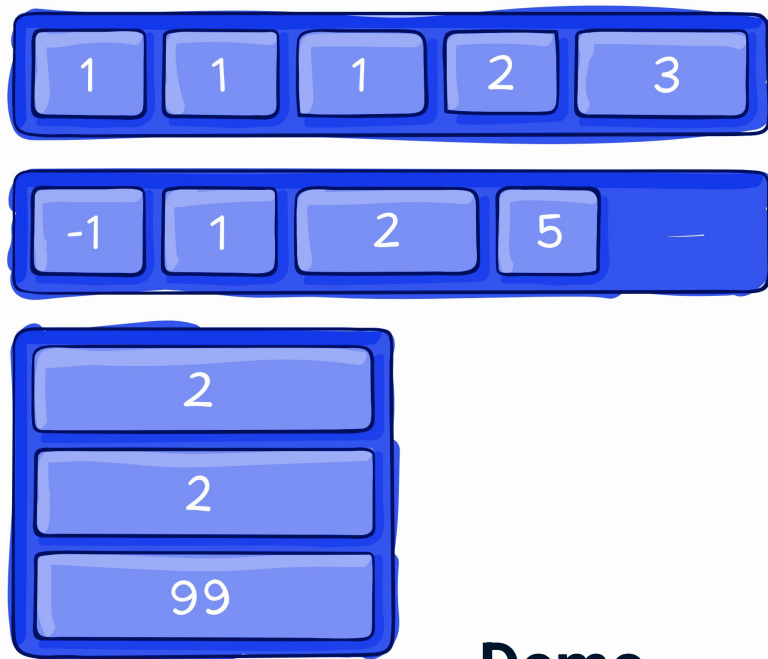
By default, flex items are laid out in the source order. However, the **order** property controls the order in which they appear in the flex container.

order: 4;

Accepts positive and negative values.

Works correctly when the **order** property is set for all elements within the flex container.

Items with the same order revert to source order.



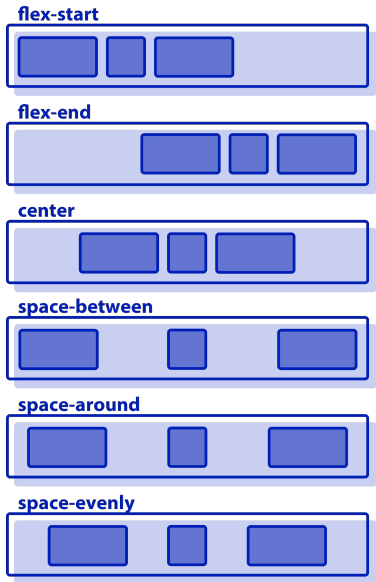
Demo

justify-content

Demo

This defines the alignment along the main axis. It helps distribute extra free space leftover when either all the flex items on a line are inflexible, or are flexible but have reached their maximum size. It also exerts some control over the alignment of items when they overflow the line.

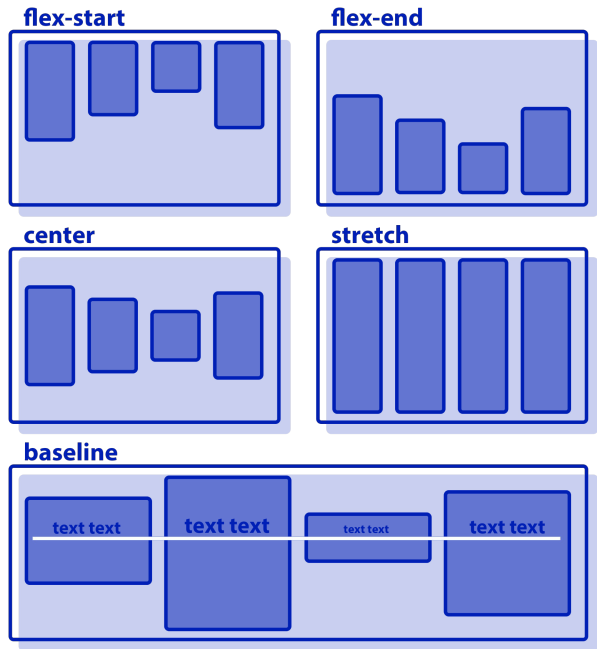
- **flex-start** (default): items are packed toward the start of the flex-direction.
- **flex-end**: items are packed toward the end of the flex-direction
- **center**: items are centered along the line
- **space-between**: items are evenly distributed in the line; first item is on the start line, last item on the end line
- **space-around**: items are evenly distributed in the line with equal space around them. Note that visually the spaces aren't equal, since all the items have equal space on both sides. The first item will have one unit of space against the container edge, but two units of space between the next item because that next item has its own spacing that applies.
- **space-evenly**: items are distributed so that the spacing between any two items (and the space to the edges) is equal.



align-items

This defines the default behavior for how flex items are laid out along the cross axis on the current line. Think of it as the justify-content version for the cross-axis (perpendicular to the main-axis).

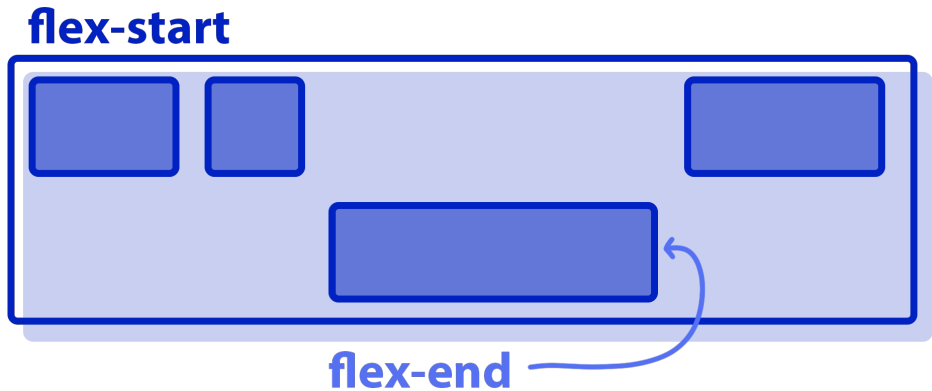
- **stretch**  (default): stretch to fill the container (still respect min-width/max-width)
- **flex-start**: items are placed at the start of the cross axis. The difference between these is subtle, and is about respecting the flex-direction rules or the writing-mode rules.
- **flex-end**: items are placed at the end of the cross axis. The difference again is subtle and is about respecting flex-direction rules vs. writing-mode rules.
- **center**: items are centered in the cross-axis
- **baseline**: items are aligned such as their baselines align



align-self

This allows the default alignment (or the one specified by align-items) to be overridden for individual flex items.

The values are the same as those for **align-items**



Demo

flex-wrap

By default, flex items will all try to fit onto one line. You can change that and allow the items to wrap as needed with this property.

Flexbox can wrap items to new rows using:

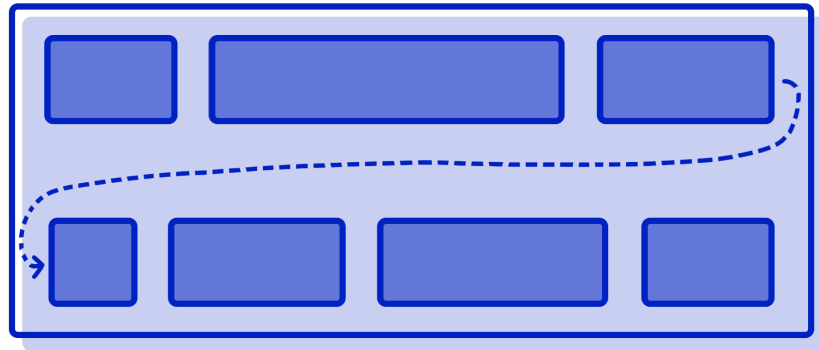
flex-wrap: wrap; 

Default (all flex items will be on one line):

flex-wrap: no-wrap; 

flex items will wrap onto multiple lines from bottom to top:

flex-wrap: wrap-reverse; 



Demo

align-content

Demo

This aligns a flex container's lines within when there is extra space in the cross-axis, similar to how justify-content aligns individual items within the main-axis.

- **flex-start** : items packed to the start of the container. The flex-start honors the flex-direction while start honors the writing-mode direction.
- **flex-end** : items packed to the end of the container. The flex-end honors the flex-direction while end honors the writing-mode direction.
- **center**: items centered in the container
- **space-between** : items evenly distributed; the first line is at the start of the container while the last one is at the end
- **space-around**: items evenly distributed with equal space around each line
- **space-evenly** : items are evenly distributed with equal space around them
- **stretch**: lines stretch to take up the remaining space



gap

– explicitly controls the space between flex items. It applies that spacing only between items not on the outer edges.

You can set **margin** for each element individually, but it's much easier to set spacing for the entire container in one place using the **flex** container.

The behavior could be thought of as a minimum gutter, as if the gutter is bigger somehow (because of something like justify-content: space-between;) then the gap will only take effect if that space would end up smaller.

It is not exclusively for flexbox, gap works in grid and multi-column layout as well.

gap – Sets spacing between items

gap : 10px 15px;

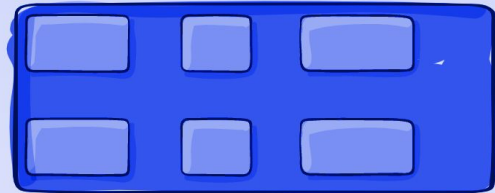
column-gap : 10px;

row-gap : 15px;

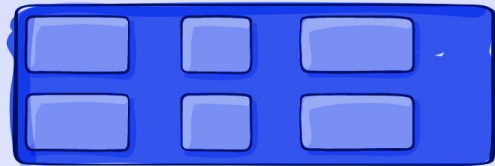
gap: 10px



gap: 30px



gap: 10px 30px



flex-grow

This defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion. It dictates what amount of the available space inside the flex container the item should take up.

Applied along the **main axis** for **flex-item**.

Values are positive integers greater than 0.

flex-grow: 2; 

The distribution is calculated based on a specific formula ->

If all items have flex-grow set to 1, the remaining space in the container will be distributed equally to all children. If one of the children has a value of 2, that child would take up twice as much of the space as either one of the others (or it will try, at least).

Demo

flex-grow calculation

$$\text{new width} = \left(\frac{\text{flex grow}}{\text{total flex grow}} \times \text{free space} \right) + \text{width}$$



flex-grow: 1 0 3

calculation:

$$\left(\frac{1}{4} \times 400 \right) + 100 \quad \left(\frac{0}{4} \times 400 \right) + 100 \quad \left(\frac{3}{4} \times 400 \right) + 100$$



flex-shrink

This defines the ability for a flex item to shrink if necessary.

Applied along the **main axis**.

Values are positive integers greater than 0.

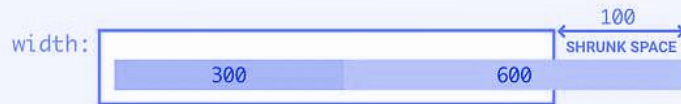
flex-shrink: 1;

The shrinkage is calculated based on a specific formula →

flex-shrink: 0; – The element will maintain the width specified by **width** or **flex-basis**, even if this causes other elements to overflow.

flex-shrink calculation

$$\text{new width} = \text{width} - (\text{shrunk space} \times \text{shrink ratio})$$



flex-shrink: 4 6

calculation:

$$\begin{aligned} \text{total shrink scaled width} &= \sum (\text{width} \times \text{flex shrink}) \\ &= (300 \times 4) + (600 \times 6) = 4800 \\ \text{shrink ratio} &= (\text{width} \times \text{flex shrink}) / \text{total shrink scaled width} \\ &= (300 \times 4) / 4800 = 0.25 \quad (600 \times 6) / 4800 = 0.75 \\ \text{new width} &= \text{width} - (\text{shrunk space} \times \text{shrink ratio}) \\ &= 300 - (100 \times 0.25) = 275 \quad 600 - (100 \times 0.75) = 525 \end{aligned}$$



flex-basis

flex-basis defines the size of an element along the **main axis** before considering **flex-grow** and **flex-shrink**.

It can be a length (e.g. 20%, 50px, etc.) or a keyword:

- The **auto** keyword means “look at my width or height property” (which was temporarily done by the main-size keyword until deprecated).
- The **content** keyword means “size it based on the item’s content” – this keyword isn’t well supported yet, so it’s hard to test and harder to know what its brethren max-content, min-content, and fit-content do.

If set to 0, the extra space around content isn’t factored in. If set to auto, the extra space is distributed based on its flex-grow value. See this graphic.

Child

flex-basis

flex-basis: auto flex-basis: auto

flex-basis: 50%

flex-basis: 50%

flex-basis: auto

flex-basis: 500px

flex-basis vs widths

flex-basis wins

```
...  
.child {  
  ✓ flex-basis: 500px  
  width: 100px  
}
```

500px

min-width wins

```
...  
.child {  
  ✓ min-width: 100px  
  flex-basis: 500px  
}
```

100px

max-width wins

```
...  
.child {  
  ✓ max-width: 700px  
  flex-basis: 500px  
}
```

700px

flex: shorthand property

flex-grow / flex-shrink / flex-basis

```
flex: 1 0 300px;
```

Flex

1	1	auto px %
---	---	-----------

Grow

Shrink

Basis

In Flexbox, **auto** margins can be used both horizontally and vertically to control the positioning of flex items within the container.

Horizontal Alignment:

- **`margin-left: auto;` or `margin-right: auto;;`**
This pushes the flex item to the opposite side, allowing for easy alignment.

Vertical Alignment:

- **`margin-top: auto;` and `margin-bottom: auto;`**
`auto;`: These work similarly to horizontal margins, pushing the item either downward (using `margin-top: auto;`) or upward (using `margin-bottom: auto;`). This is particularly useful for vertically centering items when the flex container has a set height.

Auto Margins in Flexbox



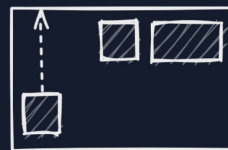
`margin-right: auto;`



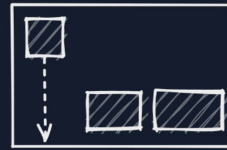
`margin-left: auto;`



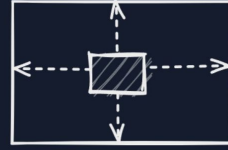
`margin-right: auto;`
`margin-left: auto;`



`margin-top: auto;`



`margin-bottom: auto;`



`margin: auto;`

Centering Content, Full-Width Background

The background of the website should always stretch across the full width of the page.

The content has a width restriction (set by width/max-width) as per the design layout and is centered on the page.

```
<section>
  <div class="content-wrapper">

section { background-color: grey; }

.content-wrapper {
  width: 1400px;
  margin-inline: auto;
}
```

Demo

